Лабораторна робота № 4

Тема. Основи об’єктно-орієнтованого програмування: класи, об’єкти та спеціальні методи.

Мета: навчитися створювати класи, працювати з об’єктами та використовувати спеціальні методи.

Короткі теоретичні відомості

Клас – це шаблон для створення об’єктів. Він визначає атрибути та методи, які будуть мати об’єкти цього типу.

Об’єкт – це конкретний екземпляр класу, створений на його основі.

Клас оголошується за допомогою ключового слова *class*, всередині якого визначаються методи та атрибути. Приклад:

```
class Student:
  def __init__(self, name, age):
    self.name = name
    self.age = age

  def show_info(self):
    print("Ім'я:", self.name, " Вік:", self.age)

student1 = Student("Петро", 25)

student1.show_info()
```

__init__() – це конструктор, тобто спеціалізований метод, який автоматично викликається під час створення нового об’єкта та ініціалізує його початковий стан.

self – це посилання на сам об’єкт.

Змінні класу – це змінні котрі оголошенні безпосередньо всередині класу, вони є спільними для всіх екземплярів.

Атрибути екземплярів – це змінні котрі оголошені через *self* у конструкторі, кожен об’єкт має свої власні значення цих атрибутів.

Корисні вбудовані атрибути класів: *__name__* – ім’я класу; *__dict__* – словник з усіма атрибутами та методами; *__doc__* – документація класу. Приклад:

```
print(Student.__name__)
print(Student.__dict__)
```

```
print(Student.__doc__)
```

Методи класу поділяється на три основні види: звичайні методи екземпляра, які працюють з конкретним об'єктом через параметр *self*; методи класу, які працюють із самим класом через параметр *cls*; статичні методи. Приклад:

```
class Student:
    university = "КрНУ"

    def __init__(self, name, age):
        self.name = name
        self.age = age

    def info(self):
        return "Студент:" + self.name + ", вік:" + str(self.age)

    @classmethod
    def change_university(cls, new_name):
        cls.university = new_name

    @staticmethod
    def is_adult(age):
        return age >= 18

student = Student("Петро", 21)
print(student.info())
Student.change_university("ЛНУ")
print(Student.university)
```

Спеціальні методи (магічні методи або dunder-методи – від double underscore) – це методи, які починаються і закінчуються двома підкресленнями (*__name__*). Головна їх особливість в тому що їх не потрібно викликати безпосередньо, а Python викликає їх автоматично, коли об'єкти бере участь у певних операціях: додаванні, відніманні, порівняння, доступі за індексом, виклику як функція і т.д.

__len__(self) – повертає кількість елементів. Приклад:

```
class Playlist:
  def __init__(self, songs):
```

```
    self.songs = songs

  def __len__(self):
    return len(self.songs)

p = Playlist(["Song1", "Song2"])
print(len(p))
```

__getitem__(self, key) – доступ за індексом/ключем. Приклад:

```
class Playlist:
  def __init__(self, songs):
    self.songs = songs

  def __getitem__(self, index):
    return self.songs[index]

p = Playlist(["Song1", "Song2"])
print(p[0])
```

*__call__(self, *args, **kwargs)* – виклик об’єкта як функції. Приклад:

```
class Multiplier:
  def __init__(self, factor):
    self.factor = factor

  def __call__(self, x):
    return x * self.factor

d = Multiplier(2)
print(d(5))
```

__eq__(self, other) – визначає чи рівні два об’єкти. Приклад:

```
class Student:
  def __init__(self, name, age):
    self.name = name
    self.age = age

  def __eq__(self, other):
    return self.name == other.name and self.age == other.age

student1 = Student("Олена", 18)
```

```
student2 = Student("Петро", 25)
print(student1 == student2)
```

Приклад реалізації декоратора класу за допомогою функції:

```
def add_common_features(cls):
    cls.version = "1.0.0"

    def info(self):
        return self.__class__.__name__ + " " + self.version

    cls.info = info
    return cls

@add_common_features
class User:
    def __init__(self, name):
        self.name = name

u = User("Петро")
print(u.info())
print(User.version)
```

Приклад реалізації декоратора класу за допомогою класу з методом *__call__*:

```
class AddInfo:
    def __init__(self, cls):
        self.cls = cls
        self.cls.version = "1.0.0"

    def __call__(self, *args, **kwargs):
        return self.cls(*args, **kwargs)



@AddInfo
class User:
    def __init__(self, name):
        self.name = name
```

```
    def info(self):
        return "Користувач: " + self.name + ", версія: "  +
self.version

u = User("Петро")
print(u.info())
print(u.version)
```

Порядок виконання роботи

1. Уважно вивчити короткі теоретичні відомості та літературу.
2. Створити власні генератори за допомогою функції з *yield* та генераторного виразу відповідно до варіанта та використати їх.
3. Написати звіт з виконання лабораторної роботи.

Варіанти завдань

№ варіанта	Назва класу	Опис
1	*BankAccount*	Клас для банківського рахунку (ім’я, баланс). Підтримує злиття рахунків та порівняння за балансом.
2	*Playlist*	Клас для керування списком пісень (назва, виконавець).
3	*Stack*	Клас, що реалізує структуру LIFO.
4	*CacheDict*	Клас-обгортка над словником, який зберігає лише останні N записів.
5	*TimeInterval*	Клас для роботи з проміжками часу (години, хвилини, секунди). Підтримує додавання, віднімання та множення на скаляр.
6	*CircularBuffer*	Кільцевий буфер фіксованого розміру. При переповненні старі дані перезаписуються.
7	*Vector3D*	Тривимірний вектор (x, y, z). Підтримує додавання, віднімання, скалярний та векторний добутки, довжину.

Контрольні питання

1. Поясніть різницю між класом та об’єктом.
2. Що таке *self* у методах класу?
3. Який спеціальний метод відповідає за ініціалізацію об’єкта і коли він викликається?
4. Чим відрізняються атрибути, визначені через *self* у конструкторі, від атрибутів, визначених безпосередньо всередині класу?
5. Для чого потрібен метод *__call__*?